TORCS AI Racing Competition

Container Setup Guide

Architecture Overview: Hybrid Local-Container Setup
You will conduct all Python code development, debugging, and AI assistance directly on your local computer using **VS Code** and **IBM Bob**. The **Podman container** acts purely as a headless simulation server that runs the TORCS game engine and streams the graphical track output directly to your web browser.

0. System Architecture & Components

Understanding how the pieces connect helps troubleshoot issues instantly during development:

1. Local Machine (Host)	Your physical PC/Mac running Python 3, VS Code, and IBM Bob. You write your AI controllers here and run the starter scripts locally.
2. Podman Engine	The container daemon running on your computer. It fetches and runs the lightweight container environment without requiring complex local dependencies.
3. TORCS Simulation Server	Runs automatically inside the Podman container on startup. It listens for control inputs on UDP port 3001 and streams telemetry back to your Python agent.
4. Web Browser Display	Provides visual rendering of the race track automatically inside any modern browser window at `http://localhost:8080`.

1. System Requirements

Requirement	Specification
Operating System	Windows 10/11, macOS 12+, or Linux (Ubuntu 20.04+)
CPU Architecture	Intel/AMD (amd64) or Apple Silicon M1/M2/M3/M4 (arm64)
Memory & Storage	8 GB RAM (16 GB recommended), 5 GB free disk space
Local Software	Python 3.9+, VS Code, IBM Bob Extension, Podman

2. Installing Podman Container Engine

Follow the instructions matching your local computer's operating system to install the container runtime.

2.1 macOS Installation

The recommended way to install Podman on Mac is using Homebrew:

```
brew install podman
podman machine init
podman machine start
```

Verify installation by running: `podman --version` in your Terminal.

2.2 Windows Installation

1. Download the Podman installer (.exe) from the official website: https://podman.io/docs/installation
2. Run the installer and ensure WSL2 (Windows Subsystem for Linux) is enabled when prompted.
3. Restart your computer if prompted by the installer.
4. Open PowerShell and initialize the Podman virtual machine:

```
podman machine init
podman machine start
```

2.3 Linux (Ubuntu/Debian) Installation

Linux runs Podman natively without needing a virtual machine initialization step:

```
sudo apt-get update && sudo apt-get install -y podman
```

3. Launching the TORCS Container Server

To run the simulation environment, execute the single command below from your host machine's terminal or PowerShell. TORCS will launch automatically inside the container on startup—no additional commands needed:

```
podman run -d --rm \
  -p 8080:8080 \
  -p 3001:3001/udp \
  --name torcs-server \
  quay.io/john_sloe/torcs-env:latest
```

Container Command Options Explained

- `-d`: Runs container in background, keeping your terminal free for local coding.
- `--rm`: Automatically cleans up container instance when stopped.
- `-p 8080:8080`: Maps web browser interface for live graphical simulation viewing.
- `-p 3001:3001/udp`: Maps local UDP communication port directly into TORCS.

Multi-Architecture Support Enabled

The container image located at `quay.io/john_sloe/torcs-env:latest` is published as a multi-arch manifest. Podman will automatically download the correct version (arm64 for Apple Silicon, amd64 for Intel/AMD) without requiring explicit flag.

4. Viewing the Live Simulation

Because TORCS starts automatically inside the container, viewing the race track is instant:

1. Open your web browser and navigate to: `http://localhost:8080`
2. You will immediately see TORCS running and paused on the standard blue status screen:

```
SCR server waiting for connection on port 3001...
```

The Blue Waiting Screen Means Ready!

When you see the blue waiting screen in your browser, the TORCS simulation server is active and actively waiting for your local Python script to connect over UDP port 3001. You are ready to launch your agent script.

5. Executing Your AI Agent Locally

With the server running and waiting on port 3001, execute your Python agent from your host machine terminal:

1. Open a terminal or PowerShell window on your host computer.
2. Navigate to your local project directory
3. Run your AI agent sample file script:

```
python3 torcs_jm_par.py
```

4. Switch back to your browser window at `http://localhost:8080`. You will see the car instantly connect and start driving on the track!

Sample files to start with
Sample file "torcs_jm_par.py" can be found here: https://github.com/ibm-ai-racing/racing-league/tree/main/installation/docker.

6. Troubleshooting & Common Issues

Symptom / Error	Probable Cause	Solution
`Connection Refused` on port 3001	Container server is stopped or port 3001/udp is blocked.	Verify container is running via `podman ps`. Rerun container startup command in Section 4.
Browser shows `Unable to connect` at port 8080	Container is stopped or port mapping failed.	Ensure port 8080 is mapped correctly (`-p 8080:8080`) when launching Podman.
Browser shows `Unable to connect` at port 6080	Container is stopped or port mapping failed.	Run `podman ps` to check if `torcs-server` is active. Restart container using command in Section 4.
Car drives off track immediately	Controller parameters aggressive for selected track.	Lower TARGET_SPEED to 60-80 km/h and increase CENTERING_GAIN in `torcs_jm_par.py`.
Podman architecture error on startup	Outdated cached image tag.	Run `podman pull quay.io/john_sloe/torcs-env:latest` to pull the updated multi-arch manifest.

7. Summary Workflow Quick Reference

Daily Development Loop

1. Start container server: `podman run -d --rm -p 8080:8080 -p 3001:3001/udp --name torcs-server quay.io/john_sloe/torcs-env:latest`
2. Open browser: `http://localhost:8080` to view the auto-started TORCS screen.
3. Edit code in local **VS Code** with **IBM Bob** assistance.
4. Run script in local terminal: `python3 torcs_jm_par.py`.
5. Stop agent (`Ctrl+C`), tweak parameters, and rerun!